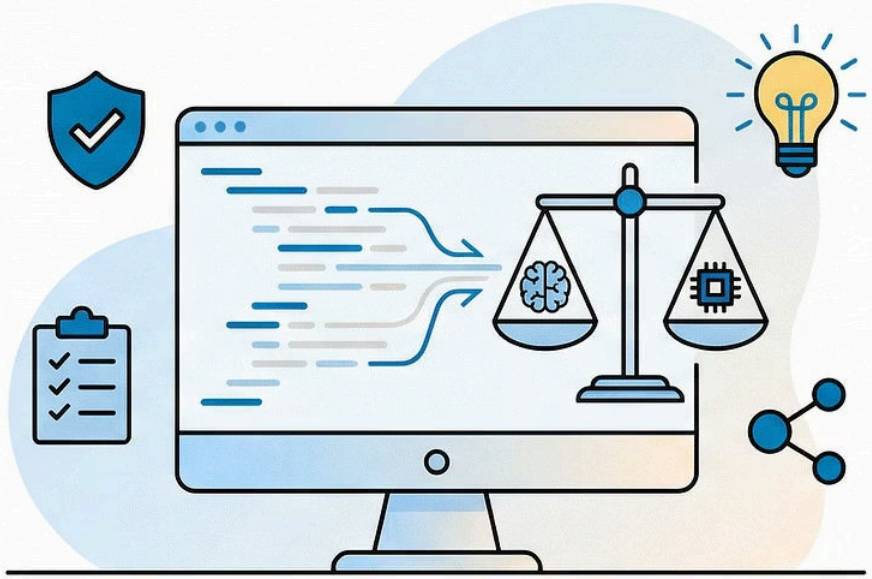
工具理性与责任主体：AI时代的编程伦理

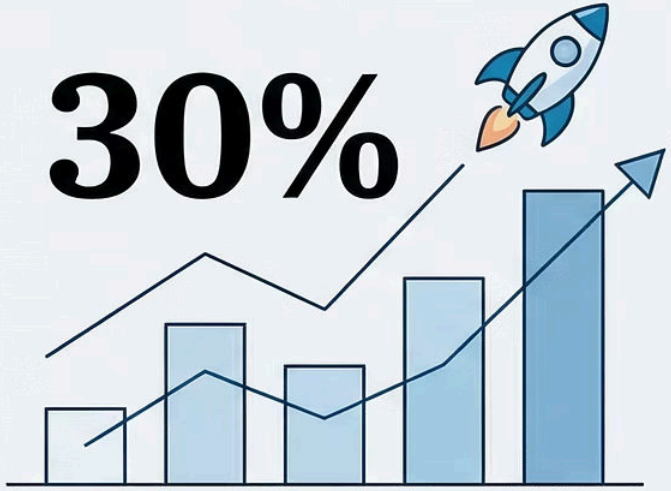
Programming Ethics in the Era of Artificial Intelligence



当AI成为编程助手，人类如何保持主导地位

30%的效率提升





- AI编程助手将开发效率提升30%，但决不能成为程序员推卸责任的借口
- 技术进步不应成为道德退步的理由
- 平衡生产力与责任感，AI是工具而非替代品，最终决策权和道德责任仍在于人

NEWS × ···

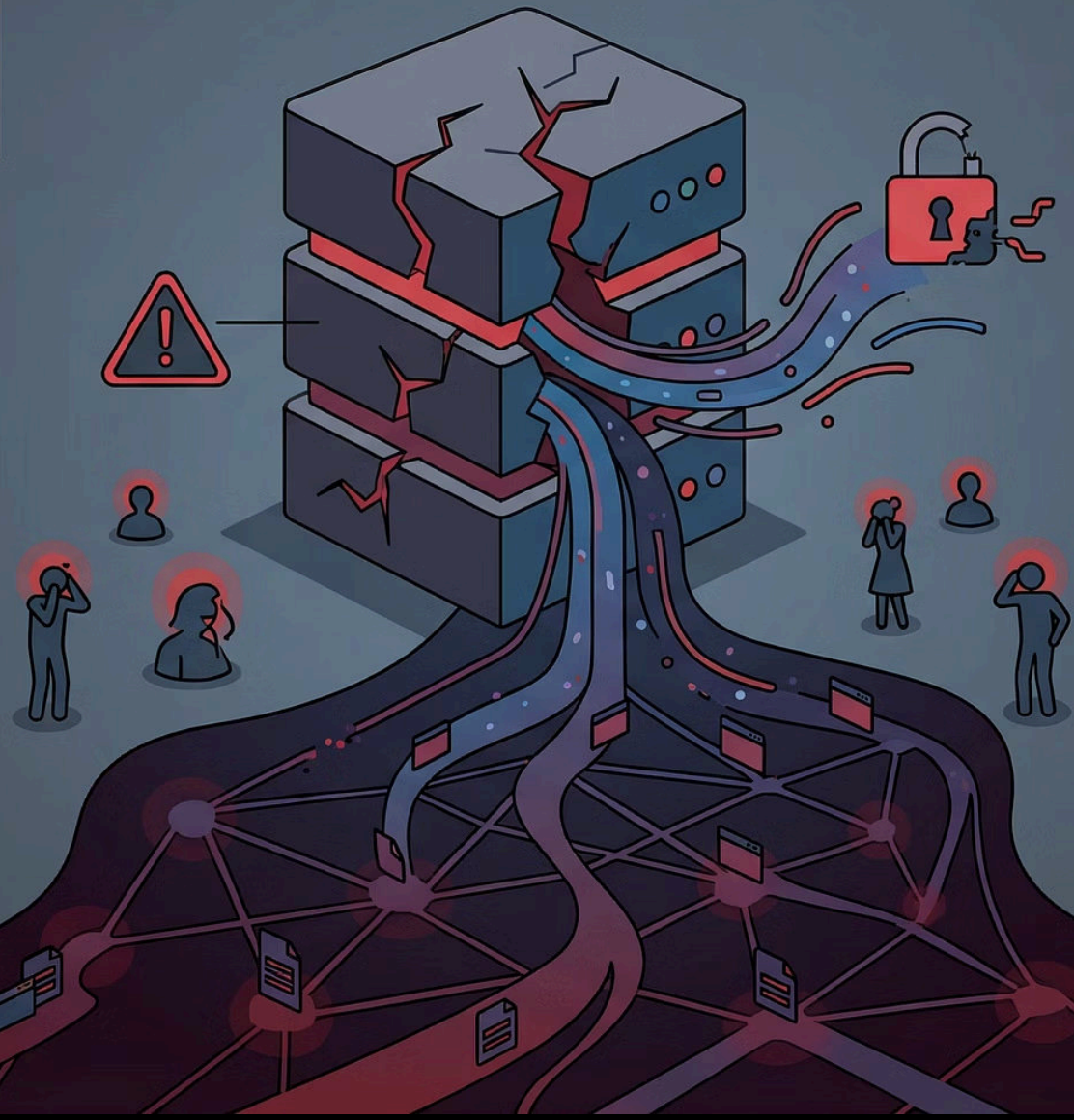
数据泄露的警示

某公司使用AI生成的数据库查询代码导致

300万

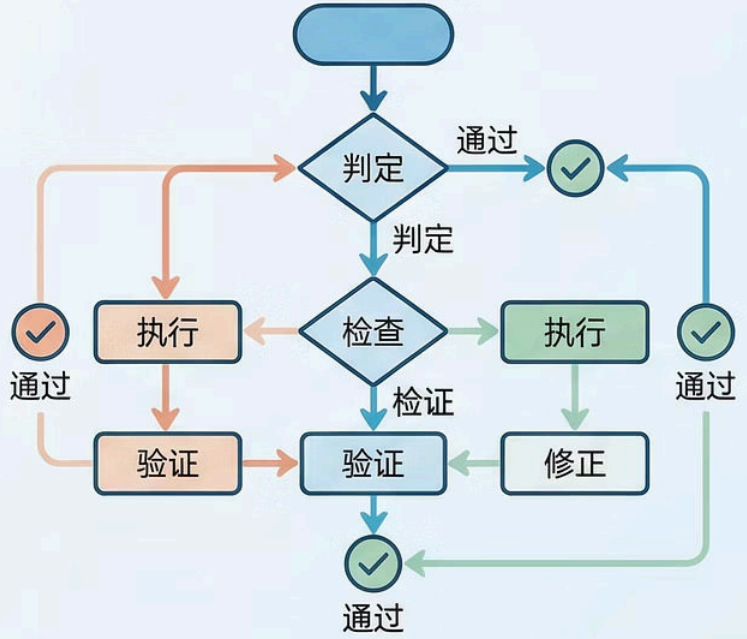
用户信息泄露

这起真实事故提醒我们：AI并非万能



统计概率vs逻辑正确





统计概率

AI基于大数据生成统计学上的概率最优解

逻辑正确

人类验证确保工程上的逻辑正确性

机长与副驾驶

‘AI是副驾驶，人是机长——这是我们必须坚守的伦理准则’



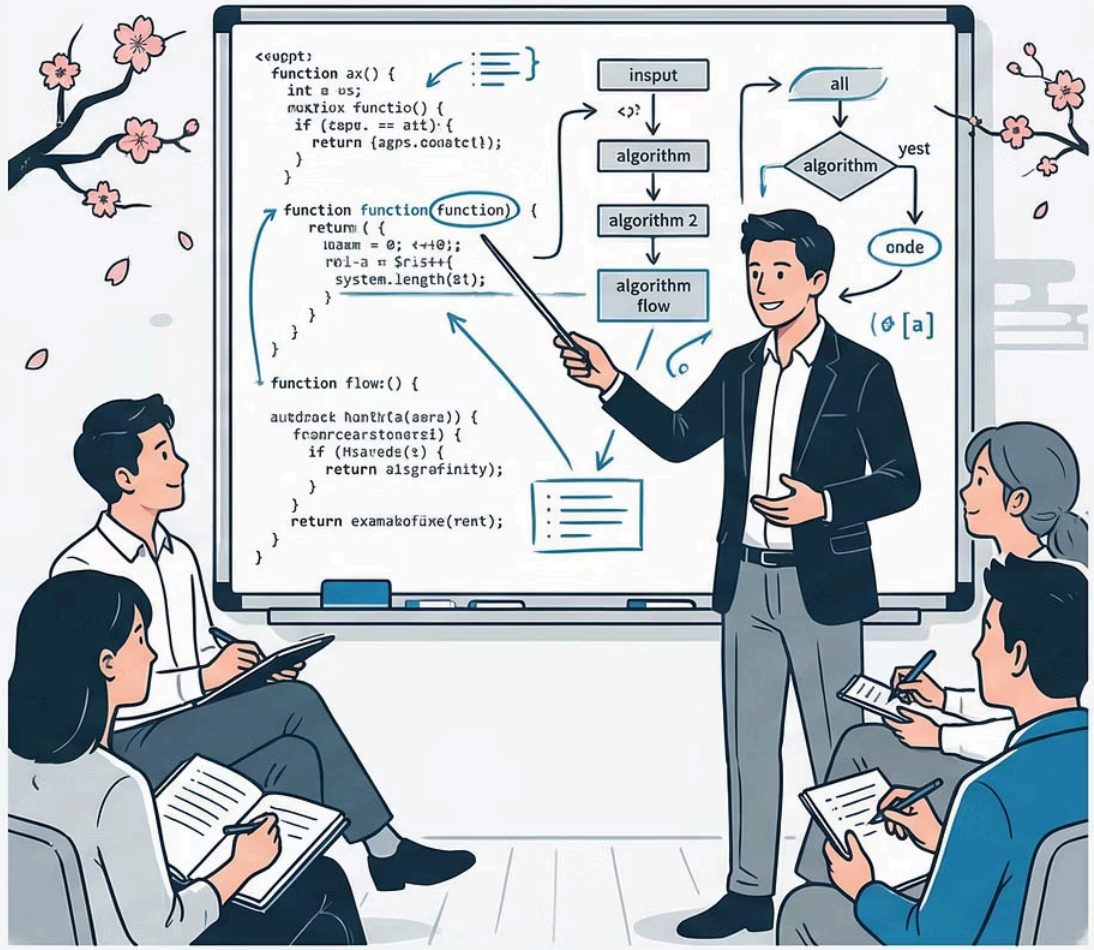
人类保持最终决策权与全局掌控力，AI提供辅助建议与效率支持

完全的解释权

完全的解释权

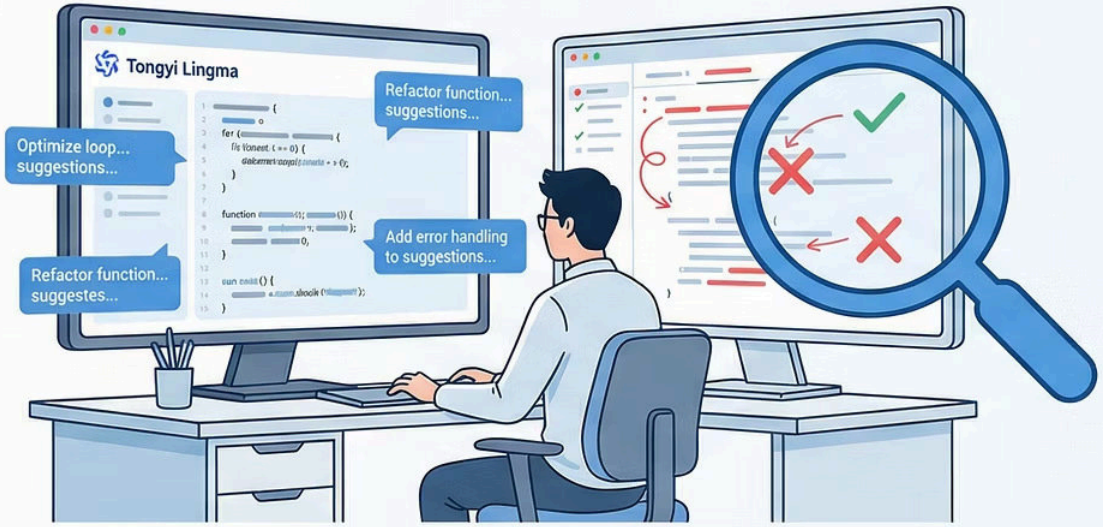
程序员必须对每一行提交的代码拥有完全的解释权与责任感

理解代码的每个细节，能够向任何人清晰解释其工作原理与潜在影响



通义灵码的批判性使用

批判性地使用通义灵码，保持独立思考能力
工具服务于人，而非人服务于工具

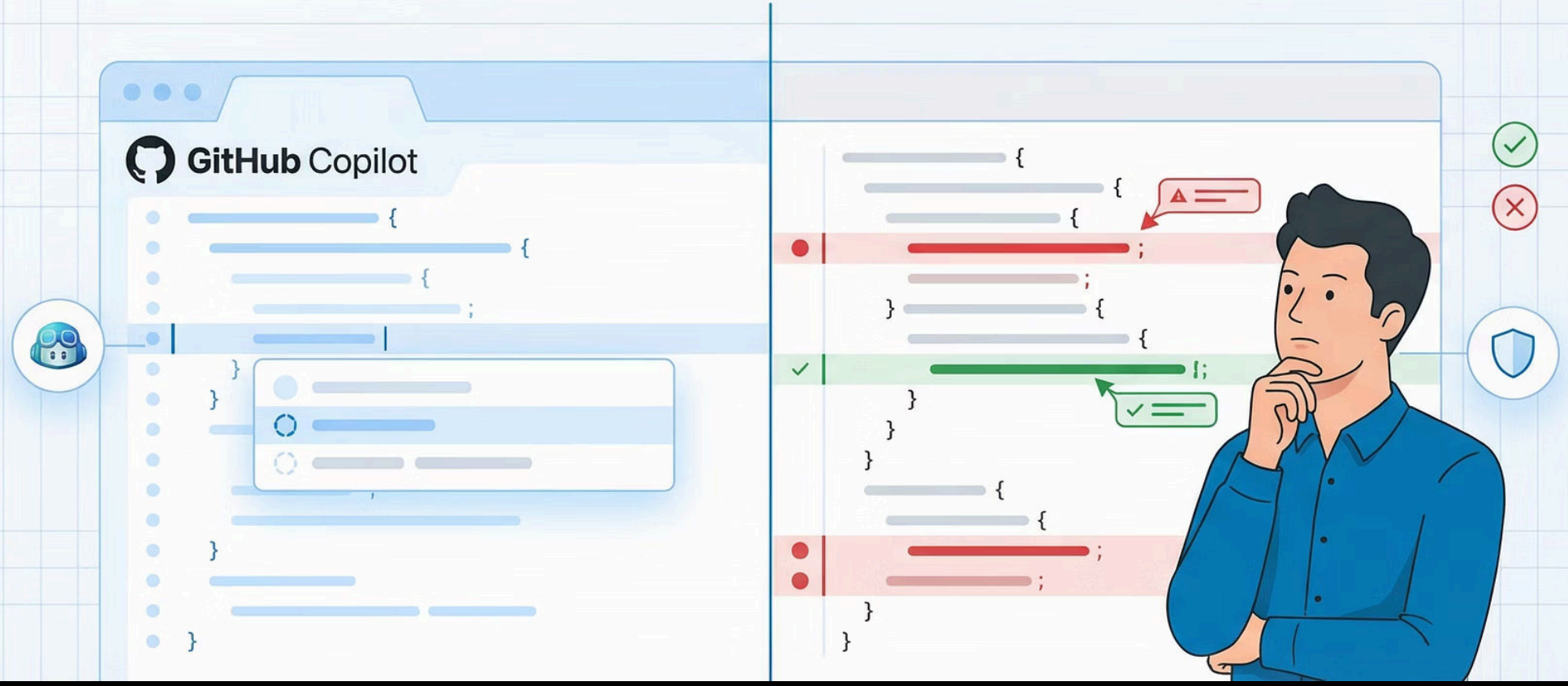


接受AI建议前进行全面评估：代码质量、
安全隐患、性能影响、维护成本

GitHub Copilot的边界

GitHub Copilot提供建议，但最终决策权在程序员手中

评估每条建议的适用性、安全性与代码质量，主动修改与完善AI生成的内容



数据主权的坚守

在享受效率红利的同时，坚守数据主权与代码安全的职业底线

保护用户数据隐私，维护代码安全标准，拒绝将敏感信息暴露给AI训练系统



核心竞争力的重新定义

程序员的核心竞争力正从





代码编写能力，向系统架构设计、代码审查和质量把控能力转变。

关注解决复杂问题的能力、技术领导力以及对业务需求的深刻理解。AI工具辅助编码，人类专注创新与高阶思维。

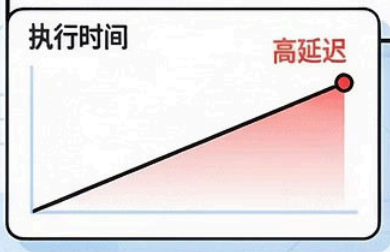
AI快速排序的陷阱

对比AI生成的朴素快速排序与人工优化的工业级代码

简单实现可能导致性能问题与边界错误，工业代码需要全面的错误处理与优化

AI快速排序（朴素实现）

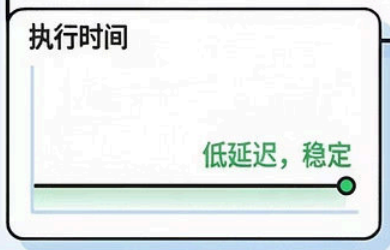
```
def quicksort(arr):
    if len(arr) < 2: return arr
    pivot = arr[0]                                  ⚠ 潜在栈溢出风险
    less = [i for i in arr[1:] if i <= pivot]       ⚠ 不稳定的性能表现
    greater = [i for i in arr[1:] if i > pivot]
    return quicksort(less) + [pivot] +              ⚠ 缺乏边界检查
           quicksort(greater)
```



人工优化（工业级代码）

```
def robust_quicksort(arr, low, high):
    if low < high:
        # 优化：三数取中法选主元                    ✓ 提高平均性能
        pi = partition(arr, low, high)
        # 优化：尾递归减少栈深度                    ✓ 避免栈溢出
        robust_quicksort(arr, low, pi - 1)
        robust_quicksort(arr, pi + 1, high)

# 增加边界检查与错误处理                            ✓ 健壮性增强
def partition(arr, low, high):
    # ...（安全、高效的实现）...
```

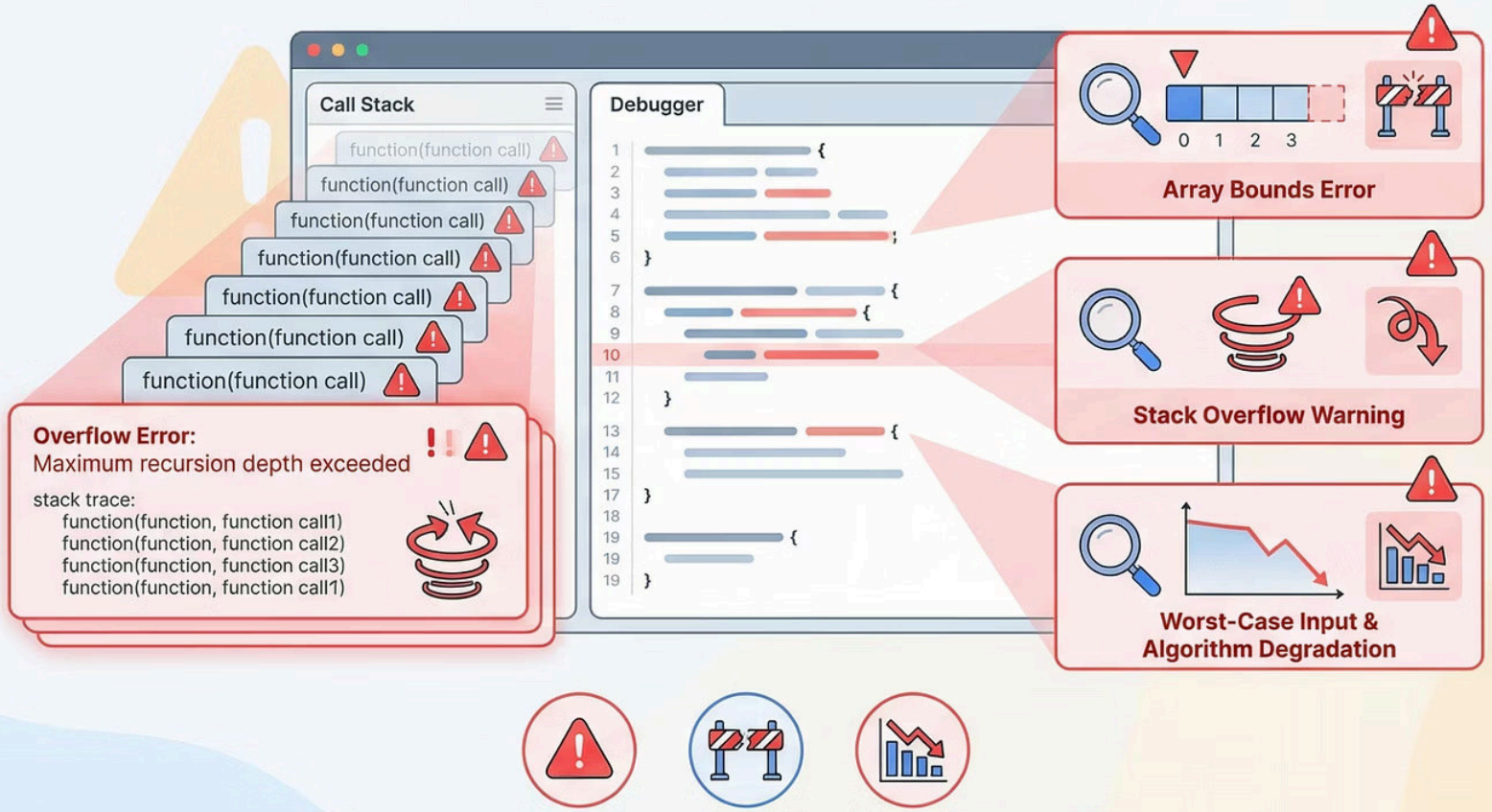




边界检查的盲区

AI在边界检查、递归深度及算法退化方面存在明显盲区

缺乏输入验证导致数组越界，无限递归导致栈溢出，特殊输入触发算法性能退化

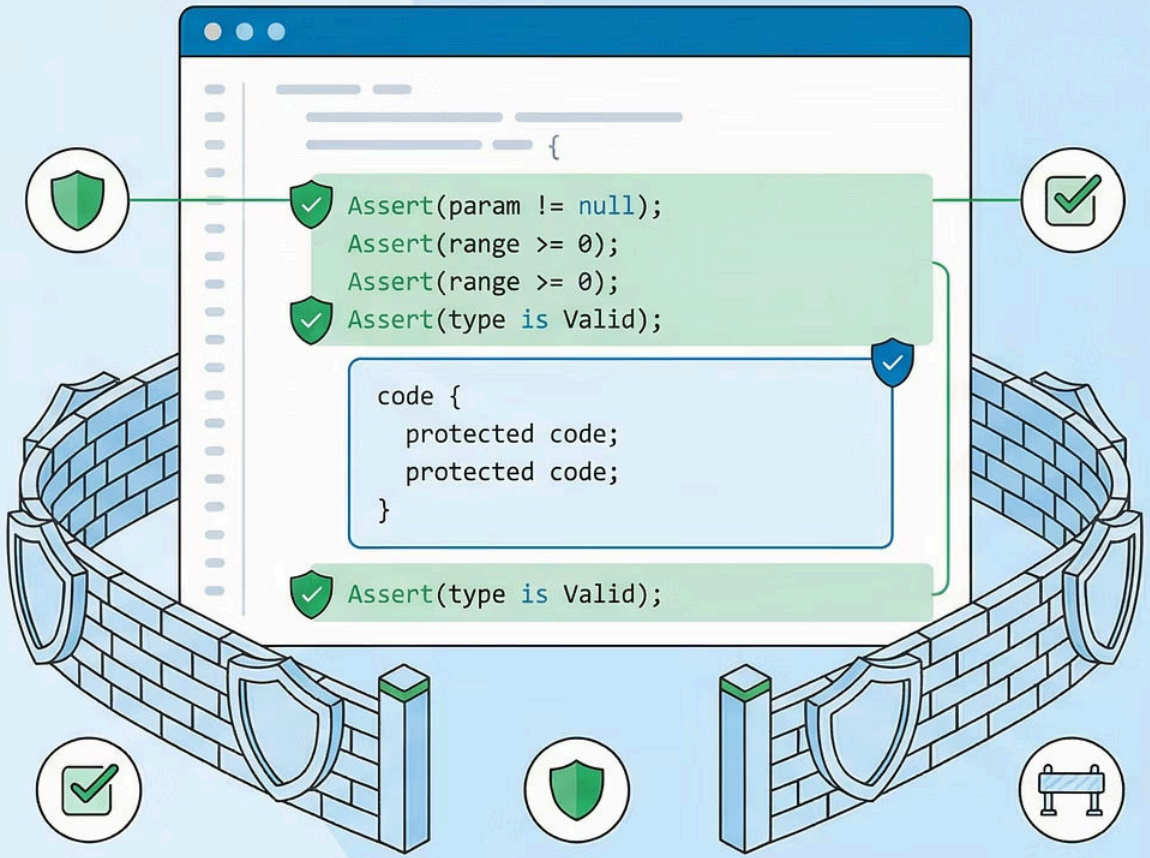


Assert断言的防护网

通过Assert断言补全AI缺失的边界防护

每个函数入口都应该有完善的参数验证

在函数开始处添加断言检查所有输入参数的合法性，及早发现错误避免系统崩溃



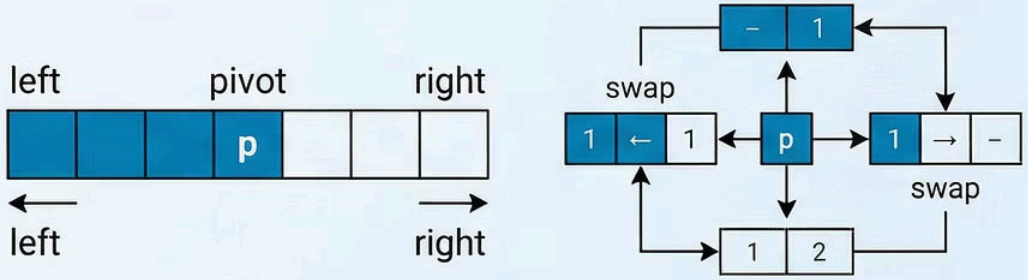
Hoare分区与尾递归优化

利用Hoare分区与尾递归消除栈溢出隐患

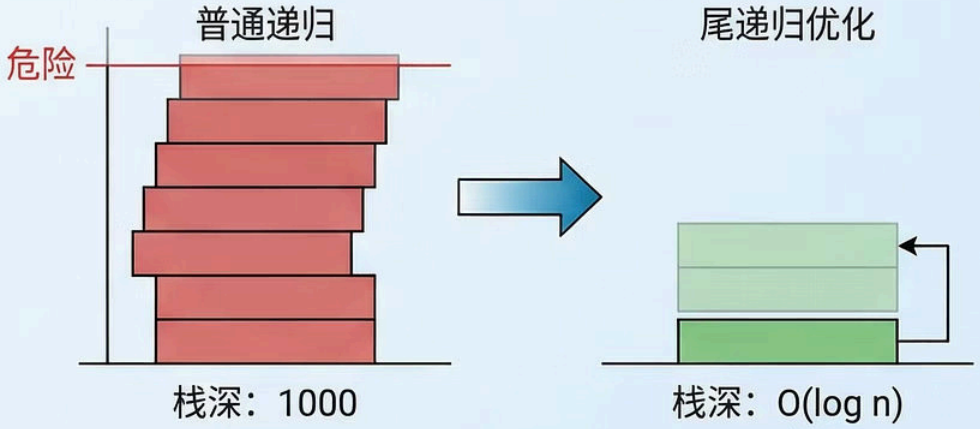
采用双向扫描分区减少交换次数，转换为尾递归形式避免栈溢出，实现O(log n)空间复杂度

算法优化是人类智慧与机器效率的完美结合

Hoare分区（Hoare Partition）



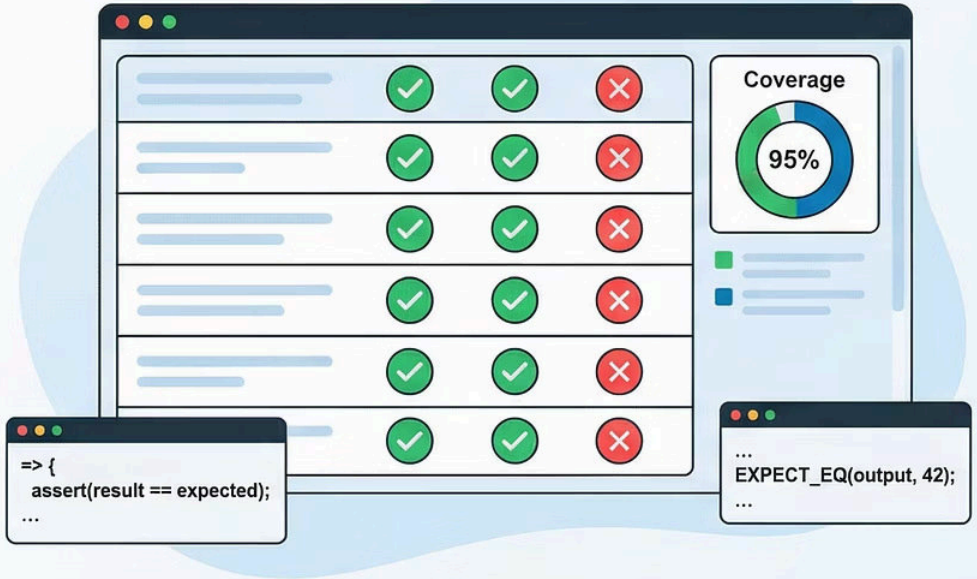
尾递归优化 (Tail Recursion Optimization)

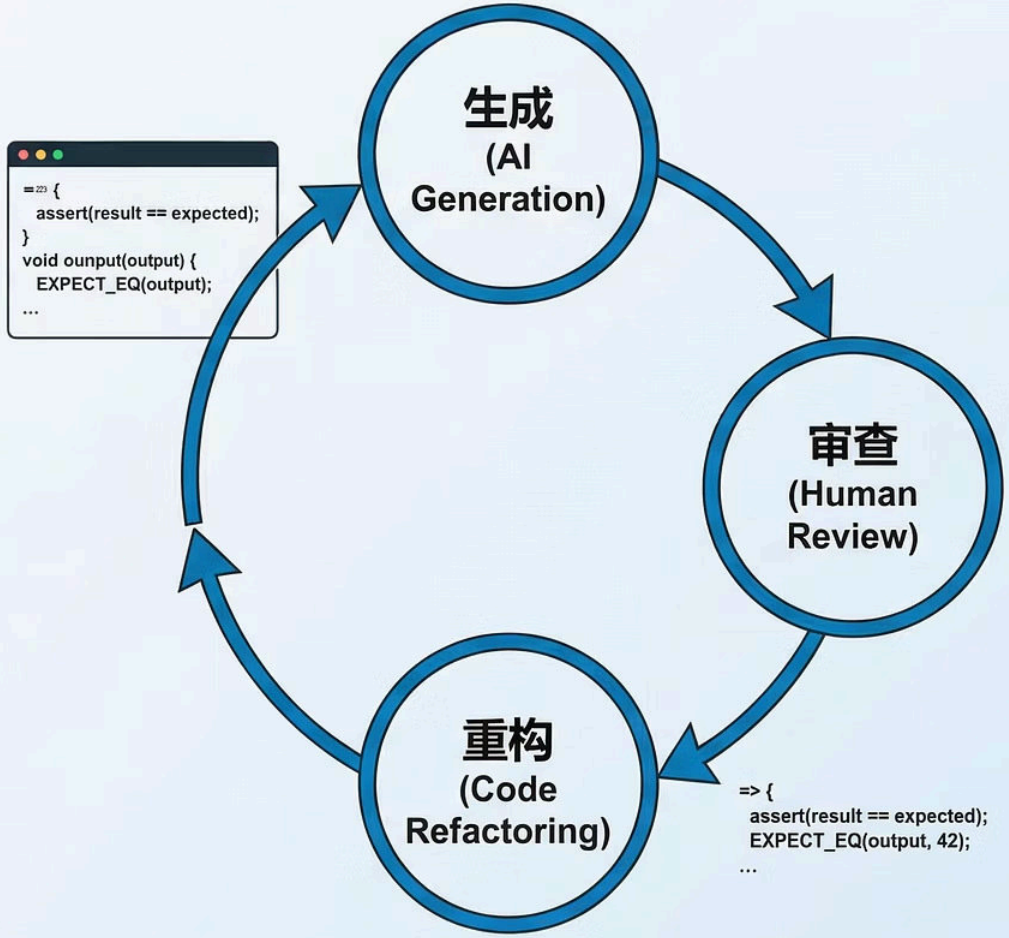


单元测试的鲁棒性验证

引入单元测试验证AI代码的鲁棒性

确保代码生成的质量与可靠性





高阶代码把关人的使命

培养既懂算法原理，又能敏锐识别机器错误的

